DIGITAL ELECTRONICS

Complete Study Notes

Number Systems · Binary Codes · Boolean Algebra · Logic Gates · Combinational & Sequential Circuits · Flip-Flops · Registers · Counters · Logic Families · SOP/POS · K-Map

Typed from the handwritten class notes — every topic, example and diagram retained, with step-by-step working added for each worked example.

Contents

1. Number System 4

1.1 Base and their group of number systems . . . 4

1.2 Minimum / maximum number of 4 digits . . . 4

1.3 Number-system conversions . . . 4

2. Binary Operations 10

2.1 Binary addition . . . 10

2.2 Binary subtraction . . . 10

2.3 Sign-magnitude representation . . . 10

2.4 1’s complement . . . 10

2.5 2’s complement . . . 11

2.6 8’s complement (octal only) . . . 11

3. Binary Codes 12

3.1 Classification of binary codes . . . 12

3.2 BCD code (Binary Coded Decimal) . . . 12

3.3 Weighted and non-weighted codes . . . 12

3.4 Excess-3 code . . . 13

3.5 Gray code . . . 13

3.6 Alphanumerical codes . . . 14

4. Important Theorems 15

4.1 Duality . . . 15

4.2 De Morgan’s theorem . . . 15

4.3 Boolean algebraic theorems . . . 15

5. Logic Gates 16

5.1 Basic gates . . . 16

5.2 Universal gates . . . 17

5.3 Special purpose gates . . . 17

5.4 Number of NAND / NOR gates needed to build each gate . . . 18

6. Combinational Circuits 19

6.1 Classification of combinational circuits . . . 19

6.2 Based on arithmetic functions . . . 19

6.3 Based on data transmission . . . 22

7. Sequential Logic Circuit 24

8. Latches and Flip-Flops 25

8.1 Flip-flop (FF) . . . 25

8.2 Race-around condition . . . 26

9. Registers 28

9.1 Types 28
10. Counter **29**
11. Logic Family **30**
11.1 Characteristics of a logic family 30
11.2 Important points 30
12. SOP and POS **32**
12.1 Express f in Sum-of-Product form (when f = 1) 32
12.2 Express f in Product-of-Sum form (when f = 0) 32
12.3 Question: expand f(A,B) = A′ + B′ in SOP and POS; find minterms and maxterms 32
13. K-Map (Karnaugh Map) **33**
13.1 K-map layouts (cell numbers = minterm numbers) 33
13.2 Rules for creating groups in a K-map 33
13.3 Reading a group (how to get the simplified term) 33
13.4 Solved K-map questions 34
13.5 Don’t-care situation 34
Appendix — Corrections and clarifications made **36**

1. Number System

1.1 Base and their group of number systems

Number System	Base	Base set (digits used)
Binary	2	0, 1
Octal	8	0, 1, 2, 3, 4, 5, 6, 7
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Hexadecimal	16	0–9 and 10 = A, 11 = B, 12 = C, 13 = D, 14 = E, 15 = F

1.2 Minimum / maximum number of 4 digits

Number System	Minimum / Maximum of 4 digits
Binary	min = 0000 (common for all systems), max = 1111
Octal	max = 7777
Decimal	max = 9999
Hexadecimal	max = FFFF

1.3 Number-system conversions

Note

Every conversion below follows the numbering of the original notes — (1) to (23). Worked examples are shown with full steps.

(1) Decimal → Binary

- Method: repeatedly divide the number by **2**, writing the remainder each time; read remainders **bottom → top**.

Example: $(25)_{10} = (?)_2$

2	**25**	1
2	**12**	0
2	**6**	0
2	**3**	1
	1	

Remainders read upward (arrow in the notes): 1 1 0 0 1

$(25)_{10} = (11001)_2$

(2) Decimal fraction → Binary

- Method: repeatedly multiply the fractional part by **2**; the integer part obtained at each step is the next binary digit (read **top → bottom**).

Example: $(0.65625)_{10} = (?)_2$

Step	Multiplication	Product	Integer part
1	0.65625 × 2	1.3125	**1**
2	0.3125 × 2	0.625	**0**
3	0.625 × 2	1.25	**1**
4	0.25 × 2	0.5	**0**
5	0.5 × 2	1	**1**

Reading the integer parts from top to bottom (arrow in the notes): **$(0.10101)_2$**

(3) Binary → Decimal

- Method: multiply each bit by its positional weight 2^n and add.

Example: $(11001)_2 = (?)_{10}$

Digit	**1**	**1**	**0**	**0**	**1**	
Weight	2^4	2^3	2^2	2^1	2^0	**Sum**
Value	16	8	0	0	1	**25**

$16 + 8 + 0 + 0 + 1 = \mathbf{(25)_{10}}$

(4) Binary fraction → Decimal

- Method: bits after the point have weights 2^{-1}, 2^{-2}, 2^{-3} …

2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}
0.5	0.25	0.125	0.0625	0.03125

Reference values of negative powers of 2 (box in the notes).

Example: $(0.10101)_2 = (?)_{10}$

Digit	**1**	**0**	**1**	**0**	**1**	
Weight	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	**Sum**
Value	0.5	0	0.125	0	0.03125	**0.65625**

$0.5 + 0 + 0.125 + 0 + 0.03125 = \mathbf{(0.65625)_{10}}$

(5) Octal → Binary

- Method: replace each octal digit by its **3-bit** binary equivalent.

Example: $(642)_8 = (?)_2$

Octal digit	6	4	2
Binary	110	100	010

$\mathbf{(642)_8 = (110100010)_2}$

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Each digit is written with 3 bits. Table at right: octal ↔ 3-bit binary.

(6) Binary → Octal

- Method: make groups of **3 digits from right to left** (pad with leading zeros), then replace each group by its octal digit.

Example: $(1011001)_2 = (?)_8$

Grouping from the right (←): 1 | 011 | 001 → pad: 001 | 011 | 001

Binary group	001	011	001
Octal digit	1	3	1

$\mathbf{(1011001)_2 = (131)_8}$

Correction to the handwritten notes

The notes show the answer as "$(31)_8$"; the leading 1 was left out. Check: $1011001_2 = 89_{10} = 131_8$ ($1\times64 + 3\times8 + 1$).

(7) Octal fraction → Binary

- Same method as octal → binary: each digit (before and after the point) becomes 3 bits.

Example: $(267.15)_8 = (?)_2$

Octal	2	6	7	.	1	5
Binary	010	110	111	.	001	101

$(267.15)_8 = (010110111.001101)_2$

(8) Binary fraction → Octal

- Same method as binary → octal.
- Digits **before** the point: group right → left. Digits **after** the point: group left → right (pad zeros on the right).

Example: $(1001101.1001)_2 = (?)_8$

Integer part ←: 001 | 001 | 101 Fraction part →: 100 | 100

Binary	001	001	101	.	100	100
Octal	1	1	5	.	4	4

$(1001101.1001)_2 = (115.44)_8$

(9) Octal → Decimal

- Method: multiply each digit by 8^n (positional weight) and add.

Example: $(7423)_8 = (?)_{10}$

Digit	**7**	**4**	**2**	**3**	
Weight	8^3	8^2	8^1	8^0	**Sum**
Value	3584	256	16	3	**3859**

3584 + 256 + 16 + 3 = **$(3859)_{10}$**

(10) Decimal → Octal

- Method: repeatedly divide by **8**; read remainders bottom → top.

Example: $(3859)_{10} = (?)_8$

8	**3859**	3
8	**482**	2
8	**60**	4
	7	

Remainders read upward: 7 4 2 3

$(3859)_{10} = (7423)_8$

(11) Octal fraction → Decimal

- Weights after the point: $8^{-1} = 0.125$, $8^{-2} = 0.015625$, $8^{-3} = 0.001953125$ (box in the notes).

Example: $(247.54)_8 = (?)_{10}$

Digit	**2**	**4**	**7**	**5**	**4**	
Weight	8^2	8^1	8^0	8^{-1}	8^{-2}	**Sum**
Value	128	32	7	0.625	0.0625	**167.6875**

128 + 32 + 7 + 0.625 + 0.0625 = **$(167.6875)_{10}$**

(12) Decimal fraction → Octal

- Integer part: divide by 8 (read upward). Fractional part: multiply by 8 (read downward).

Example: $(167.6875)_{10} = (?)_8$

8	**167**	7
8	**20**	4
	2	

Step	Multiplication	Product	Integer part
1	0.6875 × 8	5.5	**5**
2	0.5 × 8	4	**4**

Integer part: $(247)_8$ (remainders 7, 4, 2 read upward). Fraction part: $(.54)_8$

$(167.6875)_{10} = (247.54)_8$

(13) Hexadecimal → Binary

- Method: each hex digit becomes **4 binary bits**.

Hex	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Hex	Binary
8	1000
9	1001
10 = A	1010
11 = B	1011
12 = C	1100
13 = D	1101
14 = E	1110
15 = F	1111

Hex ↔ 4-bit binary table (0–7 on the left, 8–F on the right).

Example: $(7A2)_{16} = (?)_2$

Hex digit	7	A	2
Binary	0111	1010	0010

$(7A2)_{16} = (011110100010)_2$ (= 11110100010 without the leading zero)

(14) Binary → Hexadecimal

- Method: make groups of **4 bits from right to left**; replace each group by its hex digit.

Example: $(110101111)_2 = (?)_{16}$

Grouping from the right (←): 1 | 1010 | 1111 → 0001 | 1010 | 1111

Binary	0001	1010	1111
Decimal	1	10	15
Hex	1	A	F

$(110101111)_2 = (1AF)_{16}$

(15) Hexadecimal fraction → Binary

Example: $(175.5)_{16} = (?)_2$ — as written in the notes: $(172.5)_{16}$

Hex	1	7	2	.	5
Binary	0001	0111	0010	.	0101

$(172.5)_{16} = (101110010.0101)_2$

Note

The notes' question line reads "(175.5)" but the working uses the digits 1 7 2 . 5 — the working and answer correspond to $(172.5)_{16}$, which is what is shown here.

(16) Binary fraction → Hexadecimal

- Integer part: group 4 bits right → left. Fraction part: group 4 bits left → right (pad zeros on the right).

Example: $(101110010.11)_2 = (?)_{16}$

Integer ←: 0001 | 0111 | 0010 Fraction →: 11 → 1100 (pad with zeros)

Binary	0001	0111	0010	.	1100
Hex	1	7	2	.	C (=12)

$(101110010.11)_2 = (172.C)_{16}$

(17) Hexadecimal → Decimal

- Method: multiply each digit by 16^n and add.

Example: $(549)_{16} = (?)_{10}$

Digit	**5**	**4**	**9**	
Weight	16^2	16^1	16^0	**Sum**
Value	1280	64	9	**1353**

1280 + 64 + 9 = **$(1353)_{10}$**

Correction to the handwritten notes

The handwritten working has "1280 + 160 + 9 = 1449". The middle term should be 4 × 16 = **64** (not 160), so the correct answer is **1353**.

(18) Decimal → Hexadecimal

- Method: repeatedly divide by **16**; remainders 10–15 are written as A–F; read bottom → top.

Example: $(2046)_{10} = (?)_{16}$

16	**2046**	14 (E)
16	**127**	15 (F)
	7	

Remainders upward: 7, 15, 14 → 7 F E

$(2046)_{10} = (7FE)_{16}$

(19) Hexadecimal fraction → Decimal

Example: $(7EA.A)_{16} = (?)_{10}$

Digit	**7**	**E**	**A**	**A**	
Weight	16^2	16^1	16^0	16^{-1}	**Sum**
Value	1792	224	10	0.625	**2026.625**

1792 + 224 + 10 + 0.625 = **$(2026.625)_{10}$** (16^{-1} = 0.0625, so A × 0.0625 = 0.625)

(20) Decimal fraction → Hexadecimal

Example: $(2026.625)_{10} = (?)_{16}$

16	**2026**	10 (A)
16	**126**	14 (E)
	7	

Step	Multiplication	Product	Integer part
1	0.625 × 16	10	**A**

Integer part: $(7EA)_{16}$. Fraction: 0.625 × 16 = 10.000 → A.

$(2026.625)_{10} = (7EA.A)_{16}$

(21) Hexa → Octal

- Method: hex → binary (4 bits per digit), then regroup into 3 bits from the right → octal.

Example: $(8DA)_{16} = (?)_8$

Hex	8	D	A
Binary	1000	1101	1010

Regroup 100011011010 in 3s from the right (←):

Binary	100	011	011	010
Octal	4	3	3	2

$(8DA)_{16} = (4332)_8$

(22) Octal → Hexa

- Method: octal → binary (3 bits per digit), then regroup into 4 bits from the right → hex.

Example: $(377)_8 = (?)_{16}$

Octal	3	7	7
Binary	011	111	111

Regroup 011111111 in 4s from the right: 0 | 1111 | 1111 → 0000 | 1111 | 1111

Binary	0000	1111	1111
Hex	0	F (15)	F (15)

$(377)_8 = (FF)_{16}$

(23) Octal fraction → Hexadecimal

Example: $(1257.5)_8 = (?)_{16}$

Octal	1	2	5	7	.	5
Binary	001	010	101	111	.	101

Regroup in 4s: integer part ← : 0010 | 1010 | 1111 fraction part → : 1010 (pad a zero)

Binary	0010	1010	1111	.	1010
Hex	2	A (10)	F (15)	.	A (10)

$(1257.5)_8 = (2AF.A)_{16}$

2. Binary Operations

2.1 Binary addition

A	B	A + B	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1 (carry 1)

0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1, 1 + 1 = 0 (carry 1)

Example: 1011.01 + 1100.11

Add column by column from the right; carries (top row) move to the next column on the left.

carry →		1	1	1	1		1
		1	0	1	1	. 0	1
+		1	1	0	0	. 1	1
	1	1	0	0	0	. 0	0

1011.01 + 1100.11 = $(11000.00)_2$ (check: 11.25 + 12.75 = 24 = 11000_2)

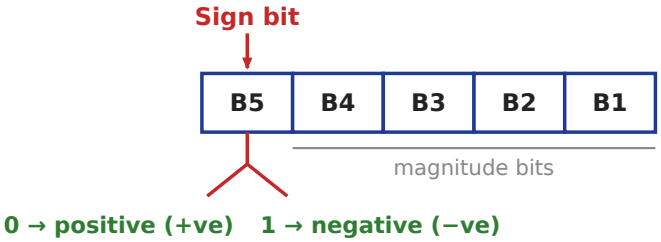
2.2 Binary subtraction

A	B	A − B
0	0	0
1	1	0
1	0	1
0	1	1 (borrow 1 from the next digit)

0 − 0 = 0, 1 − 1 = 0, 1 − 0 = 1, 0 − 1 = 1 (take a borrow from the next digit)

2.3 Sign-magnitude representation

- The **most significant bit (MSB)** is the sign bit; the remaining bits hold the magnitude.
- Sign bit **0 → positive (+ve)**, sign bit **1 → negative (−ve)**.



Sign-magnitude format (5-bit illustration: B5 is the sign bit).

2.4 1’s complement

- Invert the number (change every 0 → 1 and every 1 → 0).
- Or: subtract the number from 1 (bit-wise, i.e. from all-ones).

Example: 1’s complement of 1001101

Number	1	0	0	1	1	0	1
1’s complement	0	1	1	0	0	1	0

1’s complement = $(0110010)_2$

2.5 2’s complement

- Add **1** to the 1’s complement of the binary number.

Example: 2’s complement of 1001101

Number	1	0	0	1	1	0	1
1’s complement	0	1	1	0	0	1	0
+ 1							1
2’s complement	0	1	1	0	0	1	1

2’s complement = $(0110011)_2$

2.6 8’s complement (octal only)

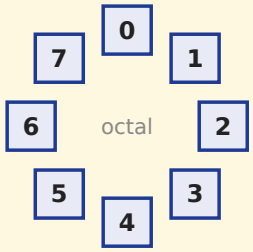
- Applies only to octal numbers.
- Method: subtract the number from 7 (digit by digit, i.e. 7’s complement) and then **add 1**.

Example: 8’s complement of 2107

Octal digits only run 0 → 7 (see the circle at the right); after 7 the count wraps to 0 with a carry, so 2107 + 1 = 2110.

	7	7	7	7
Number	2	1	0	7
7 − number	5	6	7	0
+ 1				1
8’s complement	5	6	7	1

8’s complement of $(2107)_8$ = $(5671)_8$



Adding 1 in octal: 2107 + 1 = 2110 (the carry ripples because $7 + 1 = 10_8$). The same carry rule is used when adding 1 to 5670.

Note

The handwritten heading for this item is hard to read (it looks like “8’s/8th complement”). The method written beneath it (“minus 7 and then add 1”) is the **8’s complement** — the octal counterpart of 2’s complement.

3. Binary Codes

3.1 Classification of binary codes

#	Type	Examples
1	Binary codes	BCD, Excess-3, Gray code
2	Non-binary codes	Morse code
3	Alphanumeric codes	ASCII (7-bit + 1 parity bit = 8-bit), EBCDIC (8-bit), Internal code (8-bit)

3.2 BCD code (Binary Coded Decimal)

- Binary Coded Decimal; also called the **8421 code**.
- A **4-bit** binary code.
- Decimal digits 0–9 are represented in binary (each decimal digit → its own 4-bit group).
- **Weighted code** (weights 8, 4, 2, 1).
- It is **not** a self-complementary code.
- For BCD addition: if the sum of two digits in BCD is **≤ 9**, the result is already correct; if it is **greater than 9**, add **6 (0110)** to get the correct BCD result.

Decimal to BCD conversion table

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Example: $(943)_{10} = (?)_{BCD}$

Decimal	9	4	3
BCD	1001	0100	0011

$(943)_{10} = (100101000011)_{BCD}$

Example: BCD addition rule (extra illustration — not in the original notes)

5 + 7: 0101 + 0111 = 1100 (= 12, which is > 9, so invalid in BCD). Add 0110: 1100 + 0110 = 1 0010 → **0001 0010** = BCD of 12. And 3 + 4 = 0011 + 0100 = 0111 ≤ 9, so no correction is needed.

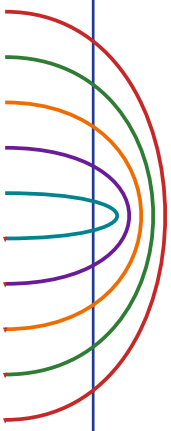
3.3 Weighted and non-weighted codes

Weighted code	Non-weighted code
Each digit position has a **fixed weight**.	Digits do **not** have any fixed weight.
e.g. BCD, 2421 code	e.g. Excess-3, Gray code

3.4 Excess-3 code

- A 4-bit code.
- **Non-weighted**.
- Obtained by **adding +3 (0011)** to the binary (BCD) code of each digit.
- **Self-complementary code** — the 9's complement of a digit is obtained by simply inverting every bit (see the arrows below).

Decimal	Binary	Excess-3
0	0000	**0011**
1	0001	**0100**
2	0010	**0101**
3	0011	**0110**
4	0100	**0111**
5	0101	**1000**
6	0110	**1001**
7	0111	**1010**
8	1000	**1011**
9	1001	**1100**



+ 0011 is added to each binary code to get Excess-3.
The coloured arcs join self-complementary pairs.

Complement pairs (9's complement): 0↔9, 1↔8, 2↔7, 3↔6, 4↔5.
e.g. Excess-3 of 3 = 0110; of 6 = 1001 — they are bit-wise inverses.

Example: $(732)_{10}$ in BCD and Excess-3

Decimal	7	3	2
BCD	0111	0011	0010

BCD = 0111 0011 0010

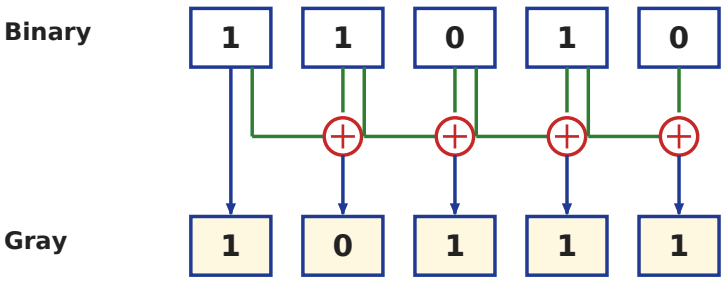
BCD	0111	0011	0010
+ 0011 each	+ 0011	+ 0011	+ 0011
Excess-3	1010	0110	0101

Excess-3 = 1010 0110 0101

3.5 Gray code

- Non-weighted code.
- Only **1 bit changes** between successive numbers — i.e. a **minimum-change code**.
- Also called: **reflected code**, **unit-distance code**, **minimum-error code**.
- Uses the **EX-OR** operation for conversion.

Rules: Binary → Gray: keep the MSB; every other Gray bit = EX-OR of that binary bit and the binary bit to its left. Gray → Binary: keep the MSB; every other binary bit = EX-OR of the Gray bit and the previous *binary* bit.



Rules for ⊕ (EX-OR):
0 ⊕ 0 = 0 0 ⊕ 1 = 1
1 ⊕ 0 = 1 1 ⊕ 1 = 0

Binary-to-Gray: first bit copied down; each next bit = EX-OR of two adjacent binary bits.

Example: Example 1: $(11010)_2 = (?)_{Gray}$

Binary	1	1	0	1	0
Operation	copy	1 ⊕ 1	1 ⊕ 0	0 ⊕ 1	1 ⊕ 0
Gray	1	0	1	1	1

$(11010)_2 = (10111)_{Gray}$

Example: Gray to Binary: $(11011)_{Gray} = (?)_2$

Gray	1	1	0	1	1
Operation	copy	1 ⊕ 1	0 ⊕ 0	1 ⊕ 0	1 ⊕ 1
Binary	1	0	0	1	0

$(11011)_{Gray} = (10010)_2$

3.6 Alphanumerical codes

- Codes that represent alphabets, numbers and punctuation marks.
- e.g. **ASCII** and **EBCDIC**.
- Normal ASCII is a **7-bit** code (with 1 parity bit it becomes 8-bit); **Extended ASCII** is an **8-bit** code.
- **ASCII** = American Standard Code for Information Interchange.
- **EBCDIC** = Extended Binary Coded Decimal Interchange Code — an **8-bit** code.

4. Important Theorems

4.1 Duality

To get the dual of a Boolean statement: interchange **AND ↔ OR** and interchange **0 ↔ 1**.

AND	↔	OR
0	↔	1
1	↔	0

Example: 1 + A = 1 has the dual 0 · A = 0.

4.2 De Morgan’s theorem

The notes list this heading only; the two standard laws are:

Law 1	(A · B)′ = A′ + B′ (NAND = bubbled OR)
Law 2	(A + B)′ = A′ · B′ (NOR = bubbled AND)

4.3 Boolean algebraic theorems

Name	AND form	OR form
Identity	1 · A = A	0 + A = A
Null	0 · A = 0	1 + A = 1
Idempotent	A · A = A	A + A = A
Inverse	A · A′ = 0	A + A′ = 1
Commutative	A · B = B · A	A + B = B + A
Associative	(A · B) · C = A · (B · C)	(A + B) + C = A + (B + C)
Distributive	A + B·C = (A + B)(A + C)	A (B + C) = A·B + A·C
Absorption	A (A + B) = A	A + A·B = A

Here A′ means the complement (NOT) of A — written with an overbar in the original notes.

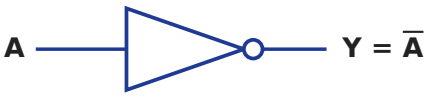
5. Logic Gates

- Logic gates are the **base of digital electronics**.
- There are **3 types**:

#	Type	Gates
(i)	Basic gates	AND, OR, NOT
(ii)	Universal gates	NAND, NOR
(iii)	Special purpose gates	EX-OR, EX-NOR

5.1 Basic gates

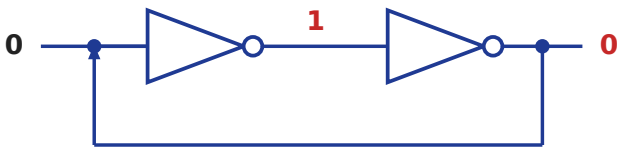
(A) NOT gate



A	A′
0	1
1	0

- Output is the inverse (complement) of the input.

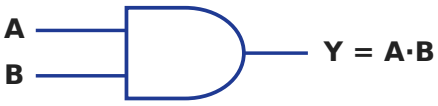
Bistable multivibrator — two NOT gates connected in a loop (output of the second fed back to the input of the first) hold either state (0 → 1 → 0 ...); this is the basic memory cell.



Output of 2nd NOT fed back to input of 1st NOT (latches the state)

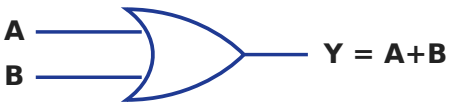
Bistable multivibrator using two NOT gates.

(B) AND gate



A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

(C) OR gate



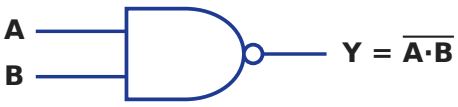
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

Remember

AND = Serial (switches in series — all must be ON), **OR = Parallel** (switches in parallel — any one ON).

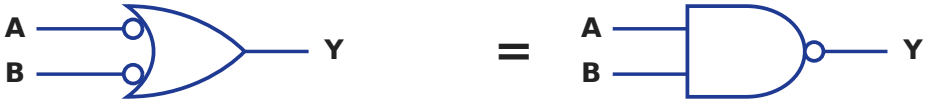
5.2 Universal gates

(A) NAND gate



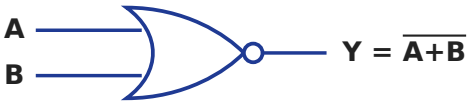
A	B	Y = (A·B)′
0	0	1
0	1	1
1	0	1
1	1	0

- NAND = AND + NOT.
- **Bubble OR gate = NAND gate** (De Morgan: A′ + B′ = (A·B)′).



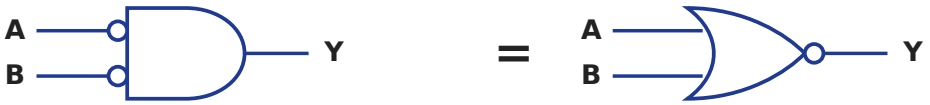
Bubble OR gate = NAND gate

(B) NOR gate



A	B	Y = (A+B)′
0	0	1
0	1	0
1	0	0
1	1	0

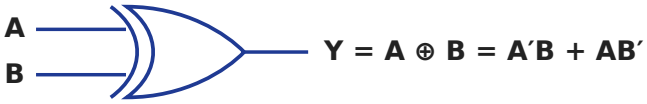
- NOR = OR + NOT.
- **Bubble AND gate = NOR gate** (De Morgan: A′ · B′ = (A+B)′).



Bubble AND gate = NOR gate

5.3 Special purpose gates

(A) EX-OR gate



A	B	Y = A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

- Output is 1 when the inputs are **different** (odd number of 1s).
- Application: **staircase switch** (a lamp controlled from two places).

(B) EX-NOR gate



Y = (A ⊕ B)′ = AB + A′B′

A	B	Y = (A ⊕ B)′
0	0	1
0	1	0
1	0	0
1	1	1

- Output is 1 when the inputs are the **same** (equality gate).

5.4 Number of NAND / NOR gates needed to build each gate

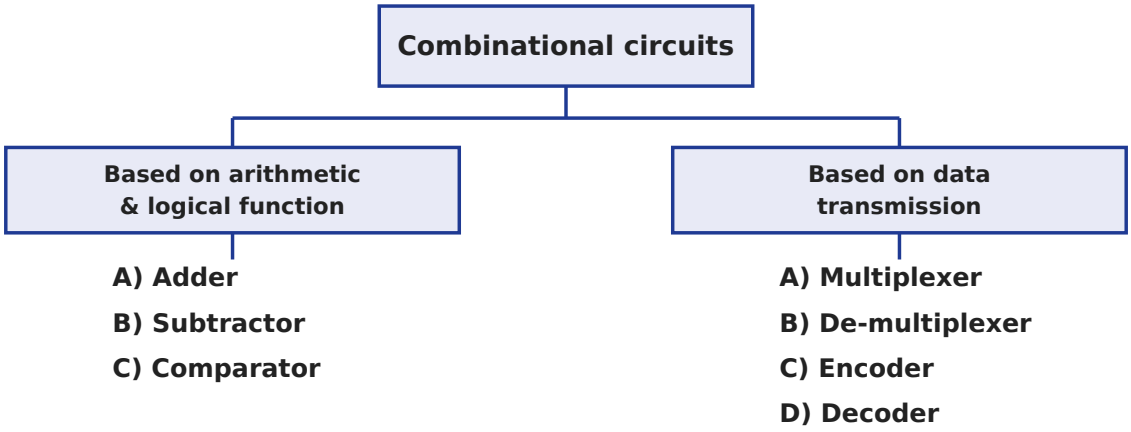
Logic gate	No. of NAND	No. of NOR
NOT	1	1
AND	2	3
OR	3	2
EX-OR	4	5
EX-NOR	5	4

Note the pattern (crossed arrows in the notes): AND ↔ OR and EX-OR ↔ EX-NOR swap their NAND and NOR counts.

6. Combinational Circuits

- Output depends on the **present input** only.
- No dependency on feedback, memory or timing.
- Built using logic gates.
- Does **not** contain any memory elements.

6.1 Classification of combinational circuits



Classification of combinational circuits.

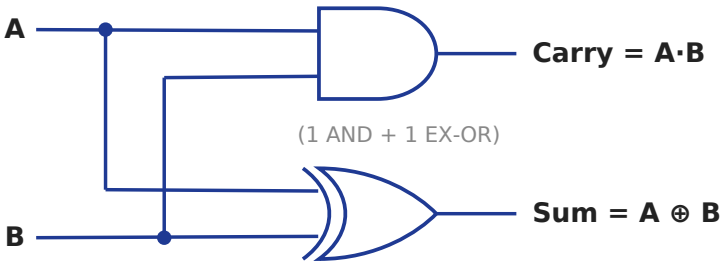
6.2 Based on arithmetic functions

(A) Adder

- Used for **binary number addition**.

(1) Half adder

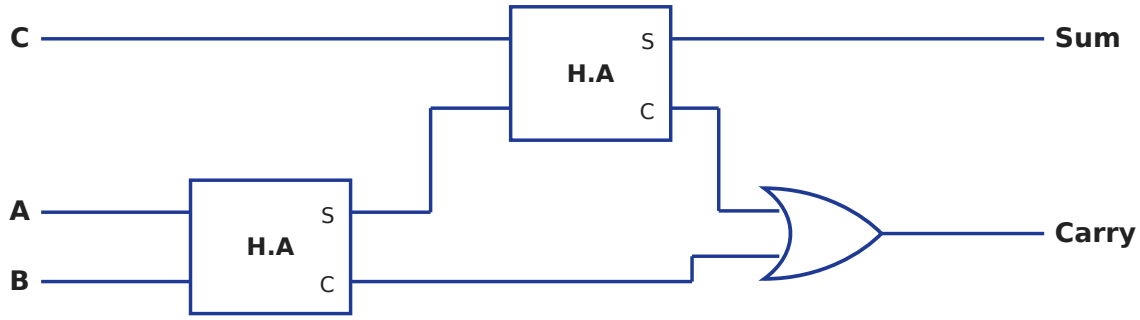
- 2 inputs & 2 outputs.
- 1 AND gate & 1 EX-OR gate.
- Output of AND = **Carry**; output of EX-OR = **Sum**.
- Can be built with **5 NAND gates** or **5 NOR gates**.



Half adder: Sum = A ⊕ B, Carry = A · B.

(2) Full adder

- 3 inputs & 2 outputs (3 inputs = 2 input bits + 1 carry-in).
- Built from **2 half adders & 1 OR gate**.
- Outputs = **Sum & Carry**.
- Can be built with **9 NAND gates** or **9 NOR gates**.



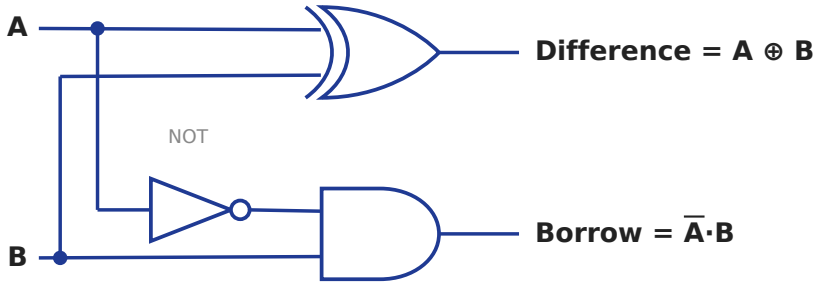
Full adder: two half adders (H.A) and an OR gate.

(B) Subtractor

- Used for **subtraction**.
- 2 outputs = **Borrow & Difference**.
- Two types: half subtractor and full subtractor.

(1) Half subtractor

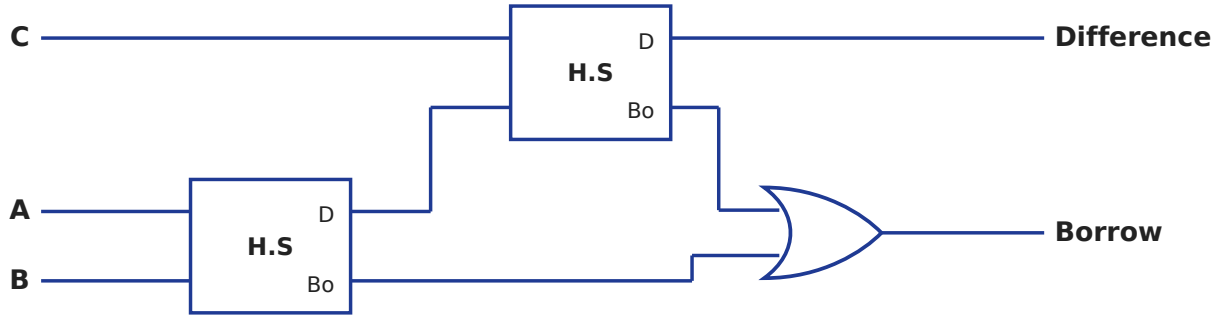
- 2 inputs & 2 outputs.
- 1 EX-OR, 1 AND, 1 NOT gate.
- Difference = A ⊕ B; Borrow = A′ · B.



Half subtractor.

(2) Full subtractor

- 3 inputs & 2 outputs.
- Built from **2 half subtractors + 1 OR gate**.
- Total 7 gates = 2 EX-OR, 2 AND & 1 OR gate (as noted) plus the 2 NOT gates inside the two half subtractors.

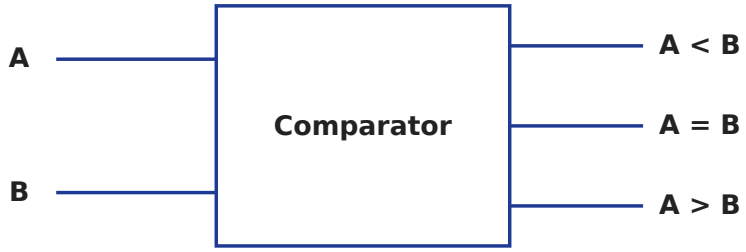


Full subtractor: two half subtractors (H.S) and an OR gate.

(C) Comparator

- A circuit that compares the given inputs and produces an output based on that comparison.
- It compares the inputs and gives output:

- 1) inputs are the same (A == B)
- 2) first is greater than the second (A > B)
- 3) first is smaller (A < B)

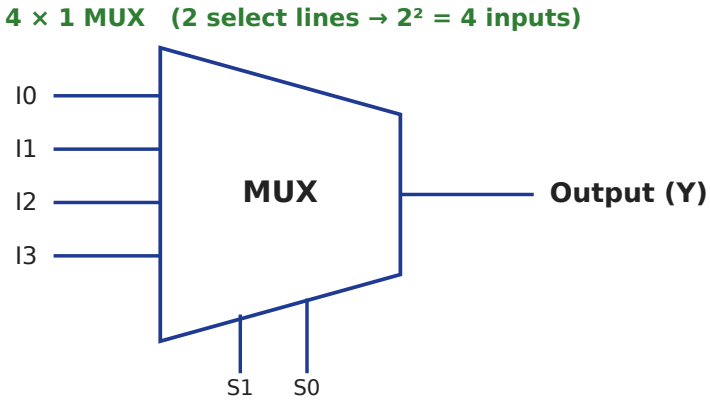


Comparator block.

6.3 Based on data transmission

(A) Multiplexer (MUX)

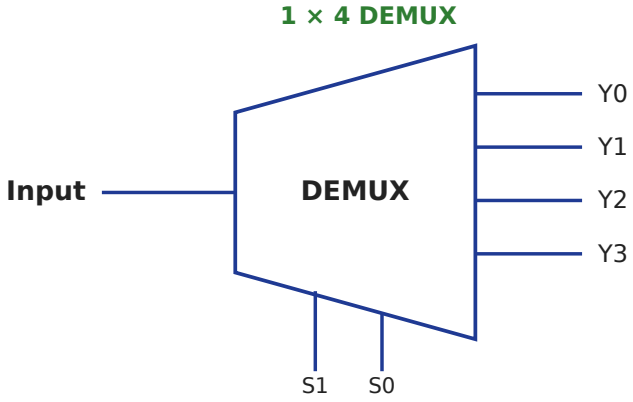
- **Many-to-one** principle: multiple inputs, one output.
- Number of inputs depends on the number of select lines: with **n select lines there are 2^n inputs**.
- Also called **Data-Selector**, because it selects only one of many inputs to pass to the output.
- **Universal circuit** — can implement many kinds of circuits.
- Applications: data selector, many-to-one, universal logic converter, parallel-to-serial converter, waveform generator.
- Types: 2×1 MUX, 4×1 MUX, 8×1 MUX.



Multiplexer block diagram (inputs, output and select lines).

(B) De-multiplexer (DEMUX)

- **One-to-many** principle (the notes say “many to one”, but a DEMUX is the opposite of a MUX).
- Reverse operation of a MUX.
- One input → many outputs; also called **data distributor**; used as **serial-to-parallel** converter.
- Relation: n = $\log_2$ m (n select lines, m outputs).
- **Cannot** be used as a universal gate.
- Types: 1×2 DEMUX, 1×4 DEMUX, etc.



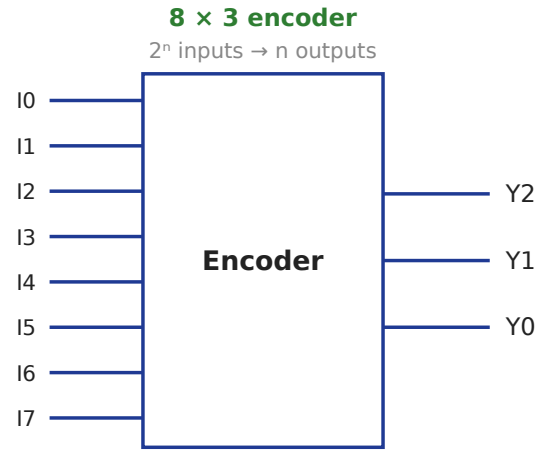
De-multiplexer block diagram.

(C) Encoder

- A circuit with multiple inputs and multiple outputs, but the number of inputs is **greater** than the number of outputs.
- Size: **2^n × n** (2^n = number of inputs, n = number of outputs).
- Used to convert various codes into binary form; also used as decimal-to-BCD encoders etc.

Example: Typical encoders

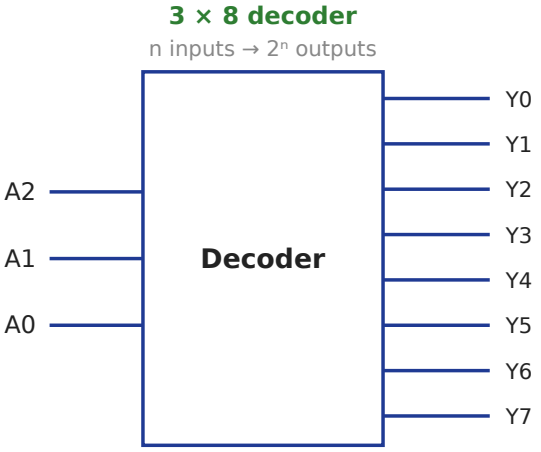
Octal to binary → **8 × 3 encoder**
Decimal to binary → **10 × 4 encoder** ($2^n = 2^4 = 16 \geq 10$, so 4 outputs are needed)
Hexa to binary → **16 × 4 encoder**



Encoder block diagram (8 × 3 shown).

(D) Decoder

- Used to perform the **reverse operation of an encoder**.
- Has multiple inputs and multiple outputs, but the number of outputs is **greater** than the number of inputs.
- Size: **n × 2^n** (n = inputs, 2^n = outputs).
- Example: **BCD decoder** — converting a binary number to decimal form.
- Types: 3 × 8 decoder, 4 × 16 decoder.



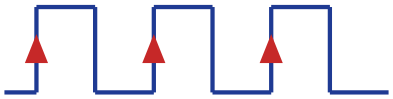
Decoder block diagram (3 × 8 shown).

7. Sequential Logic Circuit

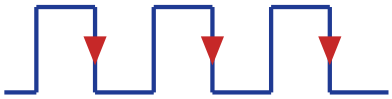
- Output depends on the **current input and the previous output / state**.
- Needs **memory** to store the previous state / information.
- **Slower** than a combinational circuit.
- Uses **feedback**.
- Two types: synchronous and asynchronous.

(1) Synchronous sequential circuit

- Their memory elements are **clocked flip-flops**.
- They are **edge triggered** (state changes only when a clock edge occurs).
- **Positive edge trigger**: the flip-flop responds on the **rising edge** of the clock (low → high).
- **Negative edge trigger**: the flip-flop responds on the **falling edge** of the clock (high → low).
- They are **slower** compared to asynchronous circuits.



Positive edge trigger (low → high)

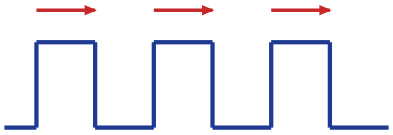


Negative edge trigger (high → low)

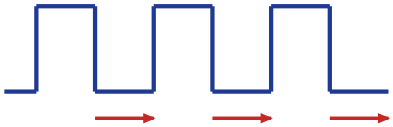
Edge triggering: arrows mark the instant at which the flip-flop responds.

(2) Asynchronous sequential circuit

- Their memory elements are **unclocked flip-flops or time delays**.
- **Level triggered**.
- Because of the absence of a clock pulse, they are **faster** than synchronous circuits.



Positive level trigger (while clock is HIGH)



Negative level trigger (while clock is LOW)

Level triggering: the circuit responds as long as the clock stays at the active level.

	Synchronous	Asynchronous
Memory element	Clocked flip-flops	Unclocked flip-flops / time delay
Triggering	Edge triggered	Level triggered
Speed	Slower	Faster (no clock pulse)

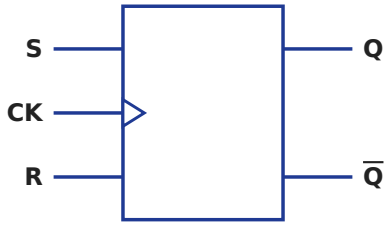
8. Latches and Flip-Flops

Latch	Flip-flop
Asynchronous input	Synchronous input
Clock pulse is **not** used	Clock pulse is used
Temporary storage	Permanent storage
Level trigger needed	Edge trigger needed
Low power consumption	More power consumption

8.1 Flip-flop (FF)

- Built from logic gates + combinational circuit.
- Uses: data transfer, data storage & frequency divider.
- 2 inputs & 2 outputs.
- Outputs: **Q** and **Q′**.
- Stores **1 bit** of data.
- Types: S-R, D, J-K, J-K master-slave, T.

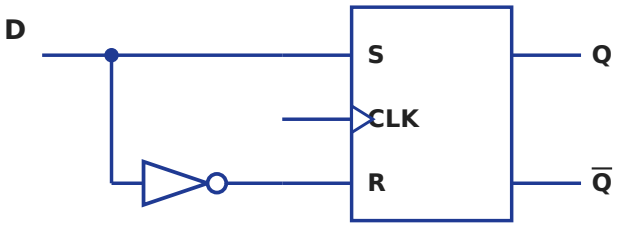
(A) S-R flip-flop



S	R	Q (next)	Action
0	0	Last Q	No change
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Not permitted

- Called a **latch flip-flop**.
- Can be made using NAND or NOR gates:
  - using NOR → **active-high** SR flip-flop
  - using NAND → **active-low** SR flip-flop
- Has two inputs: **S (Set)**, **R (Reset)**, plus the clock pulse (for a clocked SR FF).
- S = R = 1: this condition is **not permitted / invalid** (the J-K FF is used to remove it).
- S = R = 0: it **holds the previous (memory) state**.

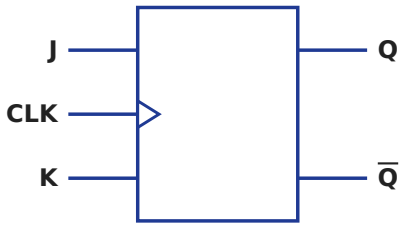
(B) D flip-flop



CLK	D	Next state (Q_{n+1})
0	X	Last state (Q_n)
1	0	0
1	1	1

- **D = Delay**.
- A modified form of the SR FF with only **one input terminal D**.
- Mainly used for building **registers**.
- Called a **Data transmission FF** or **transparent latch FF**.

(C) J-K flip-flop



CLK	J	K	Q_{n+1}
X	0	0	Q_n (prev. state)
↑	0	0	Q_n (prev. state)
↑	0	1	0 (Reset)
↑	1	0	1 (Set)
↑	1	1	Q_n' (toggle)

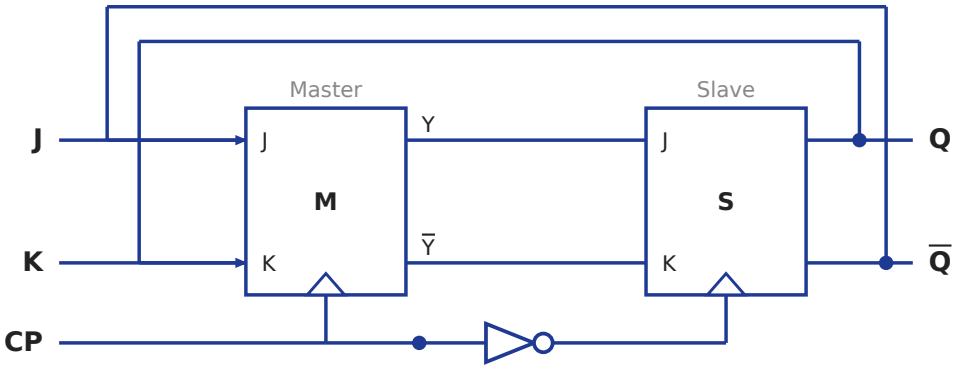
- Removes the **invalid state** of the SR flip-flop.
- Has the same **Set, Reset, No-change** conditions as the S-R FF.
- When **J = K = 1**, the output keeps toggling — called the **race / race-around condition**.
- To eliminate race-around, a **master-slave** FF is used.
- JK → Jack Kilby (or "Jump Key" FF).

8.2 Race-around condition

- Occurs in toggle mode, i.e. **J = K = 1**.
- Happens **only in level-triggered** JK flip-flops.
- Occurs when the **clock pulse width > propagation delay**.
- If $T_{pw} > T_{pd}$, the output toggles many times within one clock pulse → race-around condition.
- To avoid it: make the clock pulse width **less than** the propagation delay ($T_{pw} < T_{pd}$), or use an **edge-triggered / master-slave** flip-flop.
- T_{pw} = clock pulse width, T_{pd} = propagation delay.

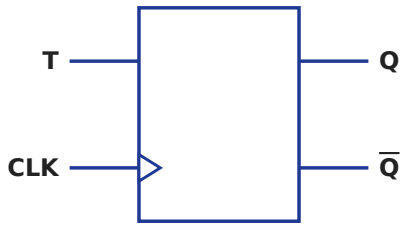
(D) J-K master-slave flip-flop

- Used to remove the race-around condition.
- Contains **two JK FFs: a master and a slave**.
- The master is triggered on the **positive edge** of the clock and the slave on the **negative edge**.
- Because the slave is edge-triggered and follows the master only on the next edge, multiple toggling during one clock pulse does not occur.
- The final output Q changes only when the slave changes, so the circuit is **stable**.



J-K master-slave flip-flop: the clock (CP) reaches the slave through a NOT gate, so master and slave work on opposite edges.

(E) T flip-flop



CLK	T	Q_{n+1} (next state)
0	0	Q_n (no change)
↑	0	Q_n (hold)
↑	1	Q_n' (toggle)

- Also called the **toggle flip-flop**.
- A modified form of the JK FF.
- Inputs J and K are **tied together** and replaced by a single input **T**.

9. Registers

- A logical circuit used to **store binary information** and control its movement.
- Built from multiple flip-flops that are logically connected.
- To store **N bits**, you need **N flip-flops**.
- Information can be stored and retrieved in **serial or parallel** form.
- Serial data is temporary, while parallel data is called positional data.

9.1 Types

(A) Buffer register

- Temporarily stores binary data.
- Implemented using **D-FF**.

(B) Shift register

- Implemented using **JK FF** (as per the notes).
- On each clock pulse, the stored data shifts by one bit.
- Formed by multiple flip-flops, all receiving the same clock.
- Types by data entry/exit:

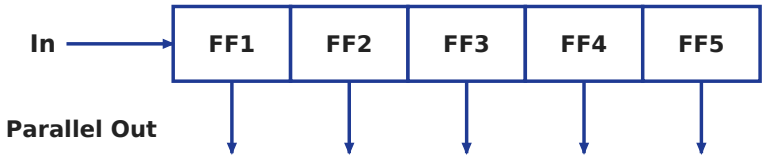
(i) SISO — Serial In Serial Out

- Data is entered into the register serially, one bit at a time.
- Stored information is taken out serially.
- Because bits go in and come out one by one, it is a **slow** type.



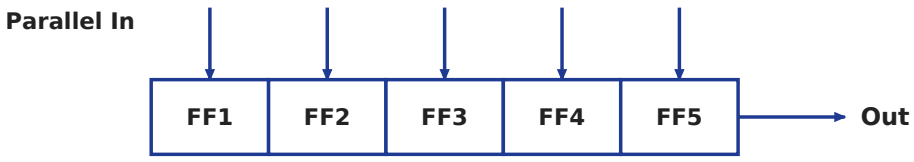
(ii) SIPO — Serial In Parallel Out

- Entered serially, taken out in parallel.



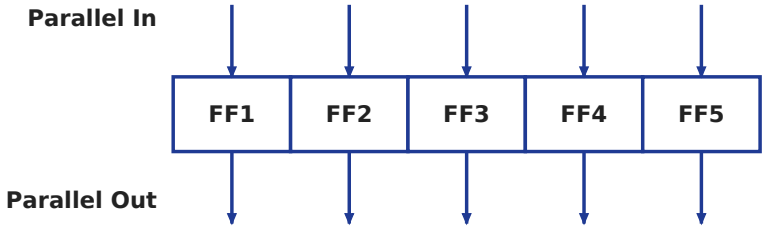
(iii) PISO — Parallel In Serial Out

- Entered in parallel, taken out serially.



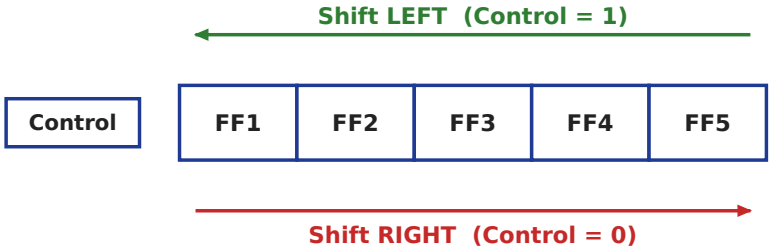
(iv) PIPO — Parallel In Parallel Out

- Entered in parallel, taken out in parallel.



(v) Bi-directional shift register

- Data can be shifted in **both directions** (left or right).
- A **control signal** selects the direction.
- Control = 0 → data shifts **right**; Control = 1 → data shifts **left**.



Bi-directional shift register.

10. Counter

- A circuit used to **count clock pulses**.
- Two types: (1) Asynchronous / Ripple counter, (2) Synchronous / Parallel counter.

(1) Asynchronous / Ripple counter

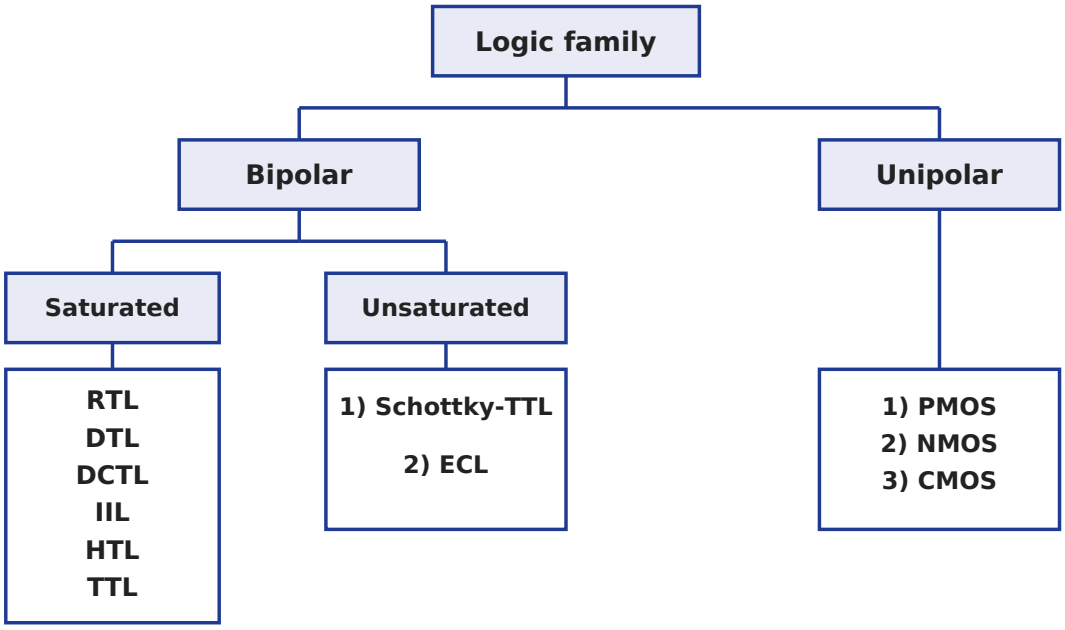
- Also called a **serial counter** because the flip-flops are arranged in serial.
- **Slower** speed.
- Only the **LSB** flip-flop receives the clock directly.
- All other flip-flops are triggered from the **output of the previous FF**.
- Triggering progresses like a **water ripple**.

(2) Synchronous / Parallel counter

- All FFs are triggered by the **same clock pulse**.
- Connected in parallel.
- **High speed**.
- More complex circuit.

	Asynchronous (Ripple)	Synchronous (Parallel)
Clock	Only LSB FF gets the clock; others from previous FF	All FFs get the same clock
Connection	Serial	Parallel
Speed	Slower	High speed
Circuit	Simple	More complex

11. Logic Family



Classification of logic families.

11.1 Characteristics of a logic family

#	Characteristic	Meaning
1	**Speed**	The time taken between a change in the input and the corresponding change in the output (propagation delay).
2	**Fan-out**	The number of loads (inputs of other gates) that the output of a gate can drive.
3	**Fan-in**	The maximum number of inputs that a logic gate can have.
4	**Power dissipation**	Energy used by each gate (measured in mW).
5	**Propagation delay**	Time taken for a signal to travel from input to output (measured in ns).
6	**Noise margin**	The maximum noise voltage that can be tolerated without causing any problem in the correct operation of the circuit.
7	**Figure of merit**	Power dissipation × propagation delay.
8	**Logic swing**	Difference of output voltages: $V_H - V_L$.
9	**Noise immunity**	The maximum noise that can be present without causing any change in the (logic) level.

11.2 Important points

- The main problem of **DCTL** (Direct Coupled Transistor Logic) is **current hogging**. To remove this problem, **IIL** (Integrated Injection Logic) is used.
- When, among several parallel lines, one line draws more current than the others, it is called **current hogging**.
- **CMOS** has the best (highest) **noise immunity**.
- **IIL** uses a **multiple-collector transistor**, which gives a **high packing density**.
- **CMOS** has the **lowest power dissipation**.

Full forms

Short form	Full form
RTL	Resistor Transistor Logic

Short form	Full form
DTL	Diode Transistor Logic
DCTL	Direct Coupled Transistor Logic
HTL	High Threshold Logic
TTL	Transistor Transistor Logic
ECL	Emitter Coupled Logic
IIL (I^2L)	Integrated Injection Logic
PMOS	P-channel Metal Oxide Semiconductor (turns ON when voltage is low/negative)
NMOS	N-channel Metal Oxide Semiconductor (turns ON when voltage is high)
CMOS	Complementary MOS (P-MOS + N-MOS together)

12. SOP and POS

SOP — Sum of Products	POS — Product of Sums
(A·B) + (B·C) + (A·C)	(A + B)·(B + C)·(A + C)
Written for the rows where **f = 1**	Written for the rows where **f = 0**
A → 1, A′ → 0	A → 0, A′ → 1
Each product term = **minterm**	Each sum term = **maxterm**
Σm = SOP	Πm = POS (Π of maxterms)

12.1 Express f in Sum-of-Product form (when f = 1)

Example: f(A,B) with truth table

	A	B	f
m0	0	0	0
m1	0	1	1
m2	1	0	1
m3	1	1	1

Take the rows where f = 1 (m1, m2, m3). In each row write 0 → A′ (complemented) and 1 → A:
01 → A′B, 10 → AB′, 11 → AB

f = A′B + AB′ + AB

This is the **standard / canonical form** (every term contains all the variables).

f(A,B) = m_1 + m_2 + m_3 = Σm(1, 2, 3) — SOP ↔ minterm

12.2 Express f in Product-of-Sum form (when f = 0)

Example: f(A,B) with truth table

	A	B	f
m0	0	0	0
m1	0	1	1
m2	1	0	0
m3	1	1	1

Take the rows where f = 0 (m0, m2). In each row write 0 → A and 1 → A′ (the opposite of SOP) and OR them:
00 → (A + B), 10 → (A′ + B)

f = (A + B)·(A′ + B) ← maxterms

f(A,B) = M_0 · M_2 = Πm(0, 2)

12.3 Question: expand f(A,B) = A′ + B′ in SOP and POS; find minterms and maxterms

Example: Solution

f(A,B) = A′ + B′ — in the first term B is missing; in the second term A is missing. Multiply each term by (missing variable + its complement):

f = A′(B + B′) + B′(A + A′)
= A′B + A′B′ + B′A + B′A′
= A′B + A′B′ + AB′ (repeated term A′B′ removed)
= 01 + 00 + 10
= m_1 + m_0 + m_2

SOP: f = Σm(0, 1, 2) — minterms m0, m1, m2

In the minterm list **m_3 is missing**, so m_3 will be the maxterm.

POS: f = Πm(3) = 11 → A′ + B′ — maxterm M3

13. K-Map (Karnaugh Map)

- Karnaugh map.
- Used to **simplify a Boolean expression** in pictorial format **without using the laws** of Boolean algebra.
- Number of cells = $\mathbf{2^n}$, where n = number of variables.

13.1 K-map layouts (cell numbers = minterm numbers)

A \ B	0	1
0	0	1
1	2	3

2 variables: $2^2 = 4$ cells

A \ BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

3 variables: $2^3 = 8$ cells

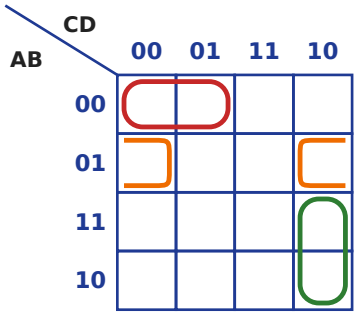
AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

4 variables: $2^4 = 16$ cells

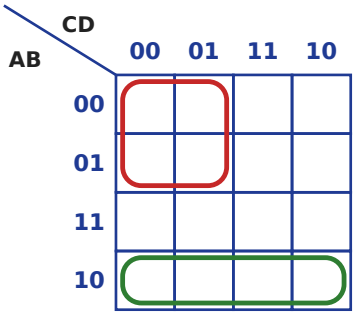
> **Note**
> Rows/columns are labelled in **Gray-code order** (00, 01, 11, 10) so that neighbouring cells differ by exactly one variable. The notes' 2-variable box shows the bottom row as "3 2"; the correct order is "2 3" (AB = 10 → m2, AB = 11 → m3), which is also what the worked Question 1 uses.

13.2 Rules for creating groups in a K-map

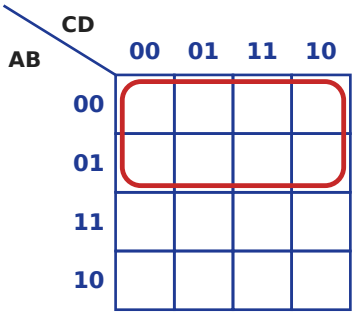
- **No zero** allowed inside a group.
- Groups can be **horizontal and vertical but not diagonal**.
- **Overlapping** is allowed.
- A group should be **as large as possible** (sizes 2, 4, 8 … cells; edges wrap around).



Pair (2 cells) — incl. wrap-around



Quad (4 cells)



Octa (8 cells)

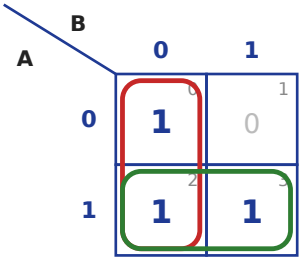
13.3 Reading a group (how to get the simplified term)

- Write the cell numbers of the group in binary.
- Variables that stay the **same** in the group are kept (1 → A, 0 → A′).
- Variables that **change** (both 0 and 1 appear) are eliminated — "cut".
- OR together the terms of all groups.

13.4 Solved K-map questions

Question 1: f(A,B) = Σ(0, 2, 3)

- 2 variables → $2^2 = 4$ cells.

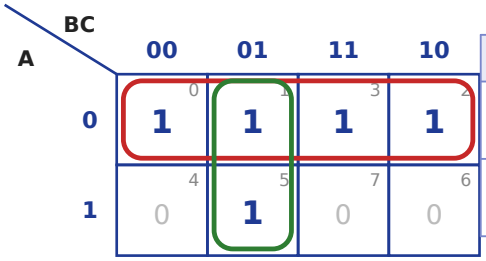


Group	Cells	A B	Result
Pair 1 (red)	m0, m2	0 0 / 1 0 → **X 0**	**B′**
Pair 2 (green)	m2, m3	1 0 / 1 1 → **1 X**	**A**

f = A + B′

Question 2: f(A,B,C) = Σ(0, 1, 2, 3, 5)

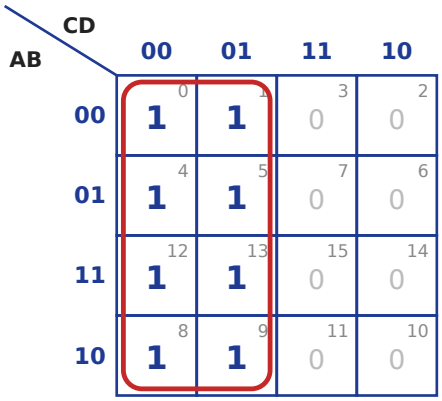
- 3 variables → $2^3 = 8$ cells.



Group	Cells	A B C	Result
Quad 1 (red)	m0, m1, m2, m3	000 001 010 011 → **0 X X**	**A′**
Group 2 (green)	m1, m5	001 / 101 → **X 0 1**	**B′C**

f = A′ + B′C (the notes call the second group “Quad 2”; it contains only the two cells m1 and m5, i.e. it is a pair).

Question 3: f(A,B,C,D) = Σ(0, 1, 4, 5, 8, 9, 12, 13)



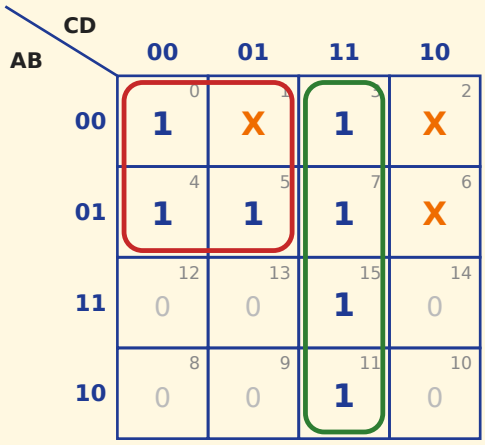
Group	A B C D (cells 0,1,4,5,8,9,12,13)	Result
Octet 1	A, B and D take both values; C is always 0 → **X X 0 X**	**C′**

f = C′

13.5 Don’t-care situation

- When the output of a Boolean expression is **unspecified**, or the output is **not needed**, the condition is called a **don’t care**.
- In K-maps it is denoted by **X**.
- In such cases, while making groups the **largest possible groups** are preferred, and X’s should be included only when they help (not more than needed) to enlarge a group.

Example: f(A,B,C,D) = Σ(0, 3, 4, 5, 7, 11, 15), d(A,B,C,D) = Σ(1, 2, 6)



Group	A B C D	Result
Quad 1 (red) m0, m1(X), m4, m5	0 0 0 0 / 0 0 0 1 / 0 1 0 0 / 0 1 0 1 → **0 X 0 X**	**A′C′**
Quad 2 (green) m3, m7, m11, m15	0 0 1 1 / 0 1 1 1 / 1 0 1 1 / 1 1 1 1 → **X X 1 1**	**CD**

The X at m1 is used in Quad 1 (it enlarges the group); the X's at m2 and m6 are not needed, so they are left out.

f = A′C′ + CD

Appendix — Corrections and clarifications made

While typing out the notes, the following small slips in the handwritten pages were corrected or clarified. Everything else is reproduced as written.

Topic	In the handwritten notes	Corrected / clarified here
Binary → Octal (6)	Answer written as $(31)_8$	$(131)_8$ (grouping 001 011 001)
Hex → Decimal (17)	1280 + 160 + 9 = 1449	1280 + 64 + 9 = **1353**
Hex fraction → Binary (15)	Question line looks like (175.5)	Working/answer are for $(172.5)_{16}$
Complement (octal)	Heading unclear	Method shown is the 8's complement
DEMUX	"many to one principle"	One-to-many principle
Full subtractor gate count	7 gates = 2 EX-OR, 2 AND, 1 OR	The remaining 2 gates are the NOT gates inside the half subtractors
K-map, 2 variables	Bottom row numbered 3, 2	Bottom row is 2, 3 (as used in Question 1)
K-map Question 2	Second group called "Quad 2"	It is a pair (m1, m5)
De Morgan's theorem	Heading only	Standard statement of the two laws added
BCD addition	Rule only	One illustrative example added (marked as extra)